Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ Информатика и системы управления

КАФЕДРА Системы обработки информации и управления

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К ЛАБОРАТОРНОЙ РАБОТЕ №3 ПО КУРСУ: РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЙ

НА ТЕМУ: Каталог одежды в стиле 12STOREEZ (вариант 13)

Выполнил студент ИБМ3-54Б _____ Михайлова В.И.
(Подпись, дата) (И.О.Фамилия)

Проверил _____
(Подпись, дата) (И.О.Фамилия)

2026 г.

ВВЕДЕНИЕ

Целью лабораторной работы является знакомство с платформой Node.js и пакетным менеджером npm, а также получение практических навыков вёрстки пользовательского интерфейса на JavaScript с использованием библиотеки Bootstrap.

В ходе работы разработано веб-приложение «Каталог одежды» из нескольких страниц: титульной страницы с названием бренда, страницы списка карточек с фильтрацией и страницы подробной информации о товаре. Данные приложение получает из mock-объектов (коллекции). Дополнительно реализованы кнопки добавления карточки (копия первой карточки) и удаления карточки, кнопка «Домой» в шапке обеих страниц, карусель карточек и всплывающие уведомления.

Интерфейс оформлен в стилистике сайта бренда 12STOREEZ: светлая минималистичная палитра, тонкий шрифт, прописные буквы с увеличенным межбуквенным интервалом, прямоугольные кнопки без скруглений. Работа выполнена по варианту 13.

ЛАБОРАТОРНАЯ РАБОТА №3. Простое веб-приложение. Верстка

Задача: знакомство с node, npm. Верстка интерфейса с карточками (страница списка с фильтрацией и страница подробнее), данные получать через mock объекты (коллекция). Добавить кнопку добавления (копировать первую карточку), кнопку удаления карточки. В хедере на обеих страницах должна быть кнопка «Домой».

Используемые технологии

- Node.js и npm – платформа и пакетный менеджер для установки зависимостей;
- Bootstrap 5 – сетка (grid), поле ввода и компоненты Carousel и Toast;
- HTML и CSS – структура страниц и собственная стилизация в стиле 12STOREEZ;
- JavaScript (ES-модули, классы) – генерация интерфейса, обработка событий и роутинг.

Подготовка проекта

Для работы установлена платформа Node.js (версия проверяется командой `node -v`). Проект создан в пустой папке lab3: командой `npm init -y` сформирован файл package.json с описанием проекта и списком зависимостей. Библиотека Bootstrap установлена командой `npm i bootstrap`: в проекте появились папка node_modules и файл package-lock.json с деревом зависимостей. Папка node_modules добавлена в .gitignore, чтобы не попадать в репозиторий.

Структура проекта приведена на рисунке 1. Код разделён по ролям: данные находятся в каталоге data, переиспользуемые части интерфейса – в components, страницы – в pages, роутер – в main.js.

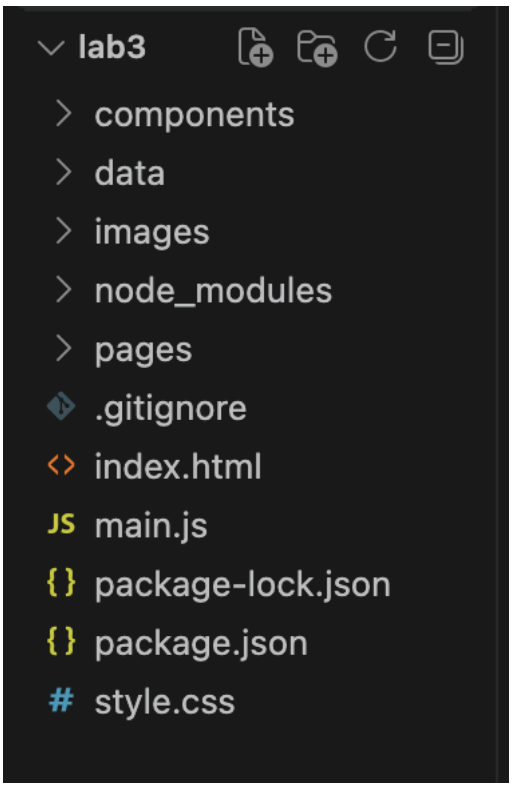


Рисунок 1 – структура проекта лабораторной №3

Файл index.html (рис. 2) содержит единственный корневой элемент `<div id="root">` и подключает стили Bootstrap из node_modules, собственные стили style.css, скрипт Bootstrap и main.js как ES-модуль (`type="module"`). Всё содержимое страниц создаётся средствами JavaScript.

index.html > ...

```
1   <!DOCTYPE html>
2   <html lang="ru">
3   <head>
4     <meta charset="UTF-8">
5     <meta name="viewport" content="width=device-width, initial-scale=1">
6     <title>12STOREEZ – каталог</title>
7     <link rel="stylesheet" href="node_modules/bootstrap/dist/css/bootstrap.min.css">
8     <link rel="stylesheet" href="style.css">
9   </head>
10  <body>
11    <div id="root"></div>
12    <script src="node_modules/bootstrap/dist/js/bootstrap.bundle.min.js"></script>
13    <script src="main.js" type="module"></script>
14  </body>
15  </html>
```

Рисунок 2 – файл index.html

Описание проекта

При открытии приложения показывается титульная страница с названием бренда (рис. 3). Кнопка «Смотреть каталог» ведёт на страницу списка. На странице списка (рис. 4) расположены шапка с кнопкой «Домой», карусель «Новинки», фильтр по категориям одежды, поле поиска, счётчик товаров, кнопка «+ Добавить» и сетка карточек. Тема карточек – женская одежда: платья, брюки, верхняя одежда, трикотаж и рубашки.



Рисунок 3 – титульная страница с названием бренда

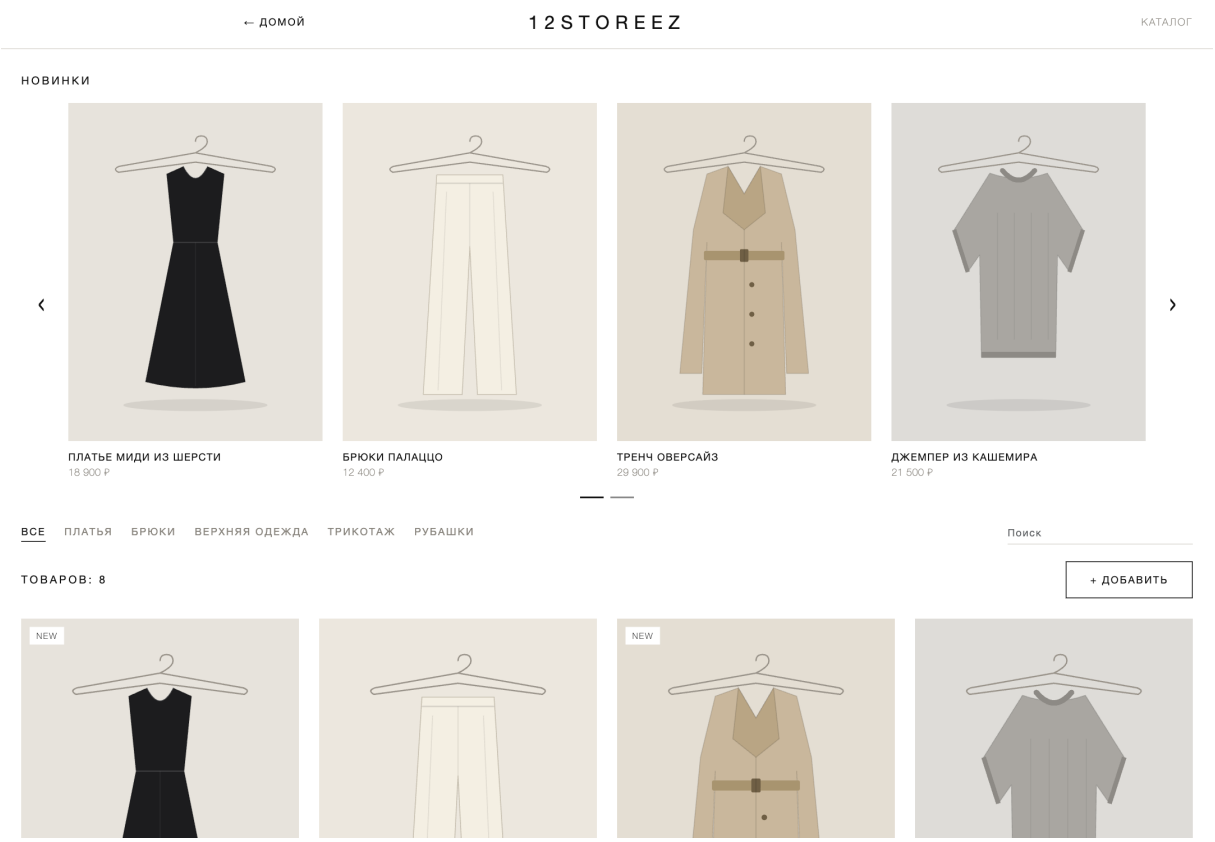


Рисунок 4 – страница списка карточек

Данные: mock-коллекция и модель

Вместо серверной части используется коллекция объектов products в файле data/products.js (рис. 5). У каждого товара есть идентификатор, название, категория, цена, цвет, состав, список размеров, метка (New / Sale), описание и изображение. Изображения хранятся в папке images и представляют собой иллюстрации изделий (одежда на вешалке) в формате SVG.

```
data > JS products.js > ...
1   // Mock-коллекция товаров (вместо backend). Изображения — SVG-заглушки.
2   const placeholder = (bg, fg, label) =>
3     'data:image/svg+xml;utf8,' + encodeURIComponent(
4       `<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 600 800"><rect width="600" height="800" fill="${bg}"/>` +
5       `<text x="300" y="410" font-family="Helvetica,Arial" font-size="26" letter-spacing="9" text-anchor="middle" fill="${fg}
6
7   export const CATEGORIES = ['Все', 'Платья', 'Брюки', 'Верхняя одежда', 'Трикотаж', 'Рубашки'];
8
9   export const products = [
10    { id: 1, title: 'Платье миди из шерсти', category: 'Платья', price: 18900, color: 'Чёрный', material: 'Шерсть 70%, вискоз
11    { id: 2, title: 'Брюки палаццо', category: 'Брюки', price: 12400, color: 'Молочный', material: 'Вискоза 55%, лён 45%', si
12    { id: 3, title: 'Тренч оверсайз', category: 'Верхняя одежда', price: 29900, color: 'Бежевый', material: 'Хлопок 100%', si
13    { id: 4, title: 'Джемпер из кашемира', category: 'Трикотаж', price: 21500, color: 'Серый меланж', material: 'Кашемир 100%
14    { id: 5, title: 'Рубашка из поплина', category: 'Рубашки', price: 8900, color: 'Белый', material: 'Хлопок 100%', sizes: [
15    { id: 6, title: 'Пальто прямого кроя', category: 'Верхняя одежда', price: 34900, color: 'Графит', material: 'Шерсть 80%,
16    { id: 7, title: 'Платье-комбинация', category: 'Платья', price: 11200, color: 'Шампань', material: 'Шёлк 100%', sizes: ['
17    { id: 8, title: 'Брюки-карго', category: 'Брюки', price: 9800, color: 'Хаки', material: 'Хлопок 100%', sizes: ['XS', 'S',
18  ];
```

Рисунок 5 – mock-коллекция товаров

Работа с коллекцией вынесена в объект store (файл data/store.js, рис. 6). Он содержит функции получения всех записей `getAll`, получения записи по идентификатору `getById`, добавления копии первой карточки `addCopyOfFirst` и удаления `remove`. При добавлении копия получает новый уникальный идентификатор (счётчик `nextId`) и вставляется сразу после первой карточки, поэтому её сразу видно на экране. Если все карточки удалены, копируется первый товар из исходной коллекции.

data > JS store.js > ...

```
1  // Модель (M в MVC): единое хранилище данных
2  import { products } from './products.js';
3
4  let items = products.map((p) => ({ ...p }));
5  let nextId = Math.max(...items.map((p) => p.id)) + 1;
6
7  export const store = {
8    getAll: () => items,
9    getById: (id) => items.find((p) => p.id === Number(id)),
10   // копия первой карточки с новым id, вставляется сразу после первой — чтобы её было видно
11   addCopyOfFirst() {
12     // если всё удалили, копируем первый товар из исходной коллекции
13     const source = items[0] ?? products[0];
14     const copy = { ...source, id: nextId++ };
15     items.splice(1, 0, copy);
16     return copy;
17   },
18   remove(id) {
19     items = items.filter((p) => p.id !== Number(id));
20   },
21 };
22
```

Рисунок 6 – модель данных store.js

Роутинг

Переключение между страницами выполняется без перезагрузки, на основе хэша в адресной строке (рис. 7). Файл main.js находит корневой элемент командой `document.getElementById('root')`, слушает событие `hashchange`, разбирает адрес и создаёт нужную страницу: адрес `#/` открывает HomePage (титульная страница), `#/catalog` – MainPage (список), `#/product/3` – ProductPage с идентификатором товара. Функция `navigate` меняет `location.hash`, что снова запускает роутер. Благодаря этому работают

кнопки «Назад» и «Вперёд» браузера, а страницу товара можно открыть по прямой ссылке.

main.js > ...

```
1   // Роутер (С в MVC): адрес в hash -> страница
2   import { HomePage } from './pages/home/index.js';
3   import { MainPage } from './pages/main/index.js';
4   import { ProductPage } from './pages/product/index.js';
5   // джава находит элемент html и нужный тэг div
6   const root = document.getElementById('root');
7   const navigate = (path) => { location.hash = path; };
8
9   function route() {
10    const path = location.hash.slice(1) || '/';
11    const match = path.match(/^\/product\/(\d+)$/);
12    let page;
13    if (match) page = new ProductPage(root, match[1], navigate);
14    else if (path === '/catalog') page = new MainPage(root, navigate);
15    else page = new HomePage(root, navigate);
16    page.render();
17    window.scrollTo(0, 0);
18  }
19
20  window.addEventListener('hashchange', route);
21  route();
22
```

Рисунок 7 – роутер в main.js

Компоненты

Интерфейс собирается из компонентов – классов JavaScript с единой схемой: конструктор принимает родительский элемент, метод `getHTML()` возвращает разметку в виде шаблонной строки, метод `render()` вставляет её на страницу через `insertAdjacentHTML` и подключает обработчики событий. В проекте созданы компоненты: ButtonComponent, HeaderComponent, FiltersComponent, ProductCardComponent, ProductComponent, CarouselComponent и функция уведомления showToast.

Компонент кнопки (рис. 8) принимает данные (идентификатор, текст, вариант оформления) и функцию-обработчик клика. Благодаря этому одна и та же кнопка используется для «+ Добавить», «Назад», «Удалить» и «Смотреть каталог».

components > button > JS index.js > ButtonComponent

```
export class ButtonComponent {
    constructor(parent) { this.parent = parent; }

    getHTML({ id, text, variant = '' }) {
      return `<button id="${id}" class="s-btn ${variant}" type="button">${text}</button>`;
    }

    render(data, listener) {
      this.parent.insertAdjacentHTML('beforeend', this.getHTML(data));
      document.getElementById(data.id).addEventListener('click', listener);
    }
  }
```

Рисунок 8 – компонент ButtonComponent

Компонент шапки (рис. 9) показывается на странице списка и на странице товара. Кнопка «← Домой» и логотип вызывают переданный обработчик, который переходит на титульную страницу (`navigate('/')`).

components > header > JS index.js > ...

```
// Шапка: логотип + кнопка «Домой» (есть на обеих страницах)
export class HeaderComponent {
    constructor(parent) { this.parent = parent; }

    getHTML(title) {
      return `
        <header class="s-header">
          <div class="inner">
            <button id="home-btn" class="s-link" type="button">← Домой</button>
            <span class="s-logo" id="logo">12Storeez</span>
            <div class="s-actions"><span class="s-link muted">${title}</span></div>
          </div>
        </header>`;
    }

    render(title, onHome) {
      this.parent.insertAdjacentHTML('beforeend', this.getHTML(title));
      document.getElementById('home-btn').addEventListener('click', onHome);
      document.getElementById('logo').addEventListener('click', onHome);
    }
  }
```

Рисунок 9 – компонент HeaderComponent с кнопкой «Домой»

Компонент карточки (рис. 10) подставляет данные товара в шаблон. Идентификатор товара записывается в атрибуты `data-id` (карточка) и `data-del` (крестик удаления). По клику обработчик определяет, куда нажал пользователь: на крестик – вызывается функция удаления, на карточку – функция открытия страницы товара.

components > product-card > JS index.js > ProductCardComponent

```
export class ProductCardComponent {
    constructor(parent) { this.parent = parent; }

    getHTML(p) {
      return `
        <div class="col-6 col-md-4 col-lg-3">
          <article class="s-card" data-id="${p.id}">
            <div class="photo"><img src="${p.image}" alt="${p.title}"></div>
            ${p.tag ? `<span class="tag">${p.tag}</span>` : ''}
            <button class="del" data-del="${p.id}" title="Удалить" type="button">×</button>
            <div class="name">${p.title}</div>
            <div class="price">${p.price.toLocaleString('ru-RU')} ₽</div>
          </article>
        </div>`;
    }

    // onOpen / onDelete получают id карточки
    render(p, onOpen, onDelete) {
      this.parent.insertAdjacentHTML('beforeend', this.getHTML(p));
      const card = this.parent.lastElementChild.querySelector('.s-card');
      card.addEventListener('click', (e) => {
        if (e.target.dataset.del) { onDelete(e.target.dataset.del); return; }
        onOpen(card.dataset.id);
      });
    }
  }
```

Рисунок 10 – компонент ProductCardComponent

Страница списка

Страница MainPage (рис. 11–13) собирает экран из компонентов: очищает корневой элемент (`innerHTML = ''`), рисует шапку, карусель, фильтры и кнопку «+ Добавить», затем вызывает `renderGrid()`. Метод `renderGrid()` берёт отфильтрованный список через свойство `filtered` и для каждого товара вызывает отрисовку карточки. Сетка перерисовывается отдельно от остальной страницы, чтобы поле поиска не теряло фокус при вводе.

```
13  export class MainPage {
14    constructor(parent, navigate) {
15      this.parent = parent;
16      this.navigate = navigate;
17    }
18
19    get filtered() {
20      const q = state.query.trim().toLowerCase();
21      return store.getAll().filter((p) =>
22        (state.category === 'Все' || p.category === state.category) &&
23        p.title.toLowerCase().includes(q));
24    }
```

Рисунок 11 – класс MainPage: конструктор и свойство filtered

```
25
26    render() {
27      this.parent.innerHTML = '';
28      new HeaderComponent(this.parent).render('Каталог', () => this.navigate('/'));
29
30      this.parent.insertAdjacentHTML('beforeend', `
31        <main class="container-fluid px-4 pb-5">
32          <div id="carousel"></div>
33          <div id="filters"></div>
34          <div class="d-flex justify-content-between align-items-center mb-4">
35            <p class="s-title" id="count"></p><div id="add-wrap"></div>
36          </div>
37          <div class="row g-4" id="grid"></div>
38        </main>`);
39
40      new FiltersComponent(document.getElementById('filters')).render(state,
41        (cat) => { state.category = cat; this.render(); },
42        (q) => { state.query = q; this.renderGrid(); });
43      new ButtonComponent(document.getElementById('add-wrap')).render(
44        { id: 'add-btn', text: '+ Добавить', variant: 'outline' },
45        () => {
46          const copy = store.addCopyOfFirst();
47          if (!copy) return;
48          // сбрасываем фильтры, чтобы новая карточка точно была видна
49          state.category = 'Все';
50          state.query = '';
51          highlightId = copy.id;
52          this.render();
53          showToast(`Добавлена копия: ${copy.title}`);
54        });
55
56      this.renderCarousel();
57      this.renderGrid();
58    }
```

Рисунок 12 – класс MainPage: метод render()

```
60  // карусель «Новинки»: первые 8 товаров, по 4 в слайде
61  renderCarousel() {
62    new CarouselComponent(document.getElementById('carousel')).render(
63      store.getAll().slice(0, 8),
64      (id) => this.navigate(`/product/${id}`));
65  }
66
67  // перерисовываем только сетку, чтобы поле поиска не теряло фокус
68  renderGrid() {
69    const grid = document.getElementById('grid');
70    const items = this.filtered;
71    grid.innerHTML = '';
72    document.getElementById('count').textContent = `Товаров: ${items.length}`;
73    if (!items.length) { grid.innerHTML = '<div class="s-empty">Ничего не найдено</div>'; return; }
74    const card = new ProductCardComponent(grid);
75    items.forEach((p) => card.render(p,
76      (id) => this.navigate(`/product/${id}`),
77      (id) => { store.remove(id); this.renderCarousel(); this.renderGrid(); showToast('Карточка удалена'); }));
78    if (highlightId) {
79      grid.querySelector(`[data-id="${highlightId}"]`)?.classList.add('is-new');
80      highlightId = null;
81    }
82  }
```

Рисунок 13 – класс MainPage: методы renderCarousel() и renderGrid()

Фильтрация и поиск

Состояние фильтра (выбранная категория и поисковая строка) хранится в объекте state вне класса страницы, поэтому сохраняется при возврате со страницы товара. Свойство `filtered` оставляет только товары, подходящие под выбранную категорию и содержащие введённый текст в названии. Результат фильтрации по категории «Платья» показан на рисунке 14, результат поиска по слову «тренч» – на рисунке 15.

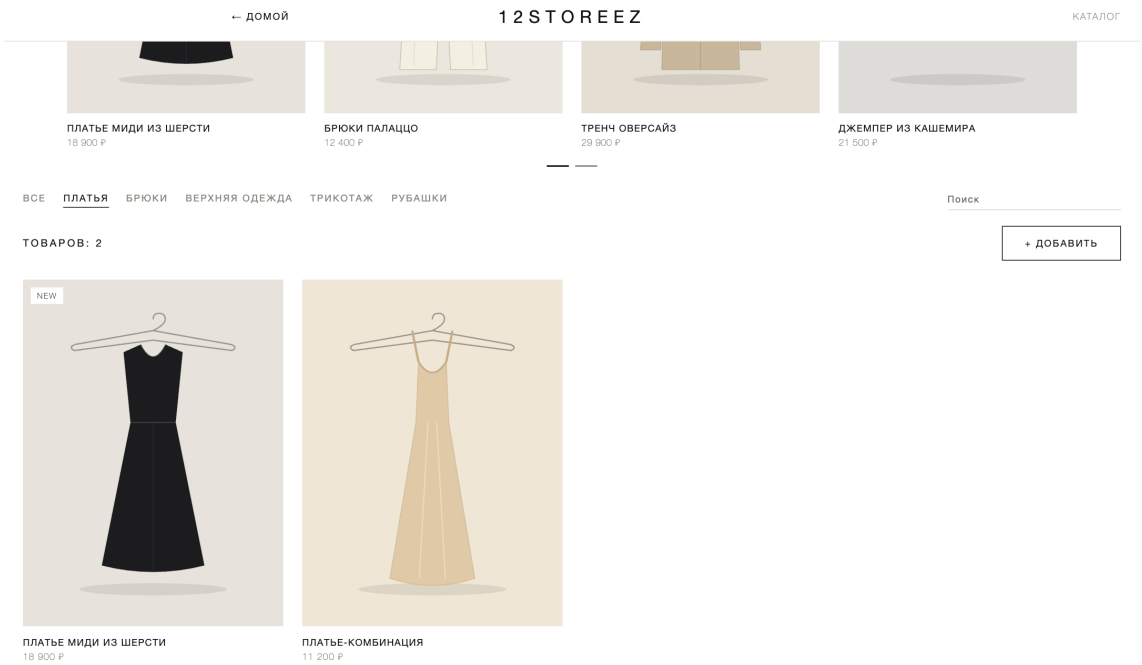


Рисунок 14 – фильтрация по категории «Платья»

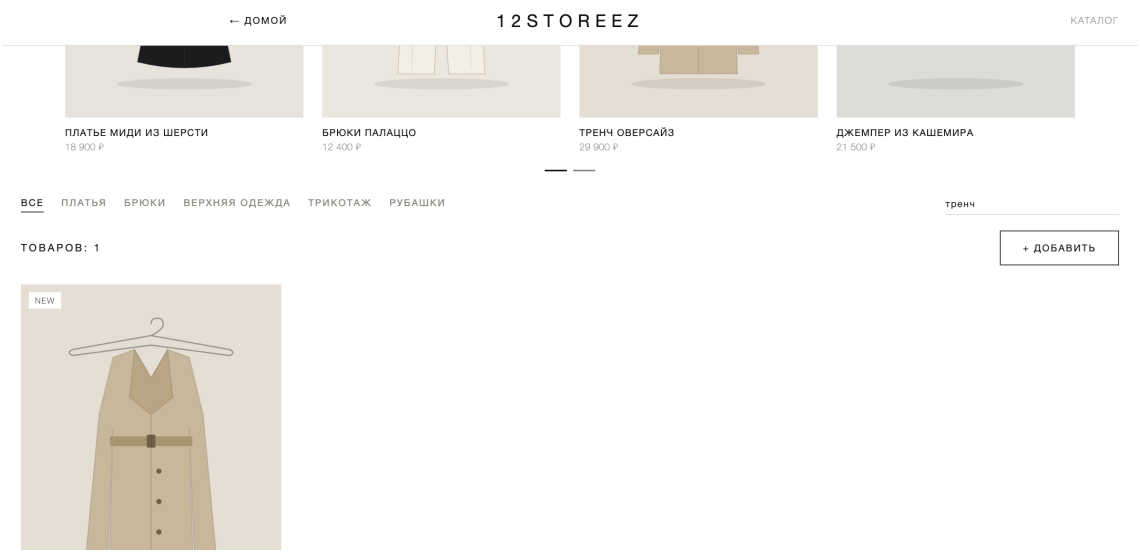


Рисунок 15 – поиск по названию

Добавление и удаление карточек

Кнопка «+ Добавить» вызывает `store.addCopyOfFirst()`, сбрасывает фильтры, запоминает идентификатор новой карточки, перерисовывает страницу и показывает уведомление (рис. 16). Новая карточка выделяется рамкой. Количество товаров в счётчике увеличилось с 8 до 9.

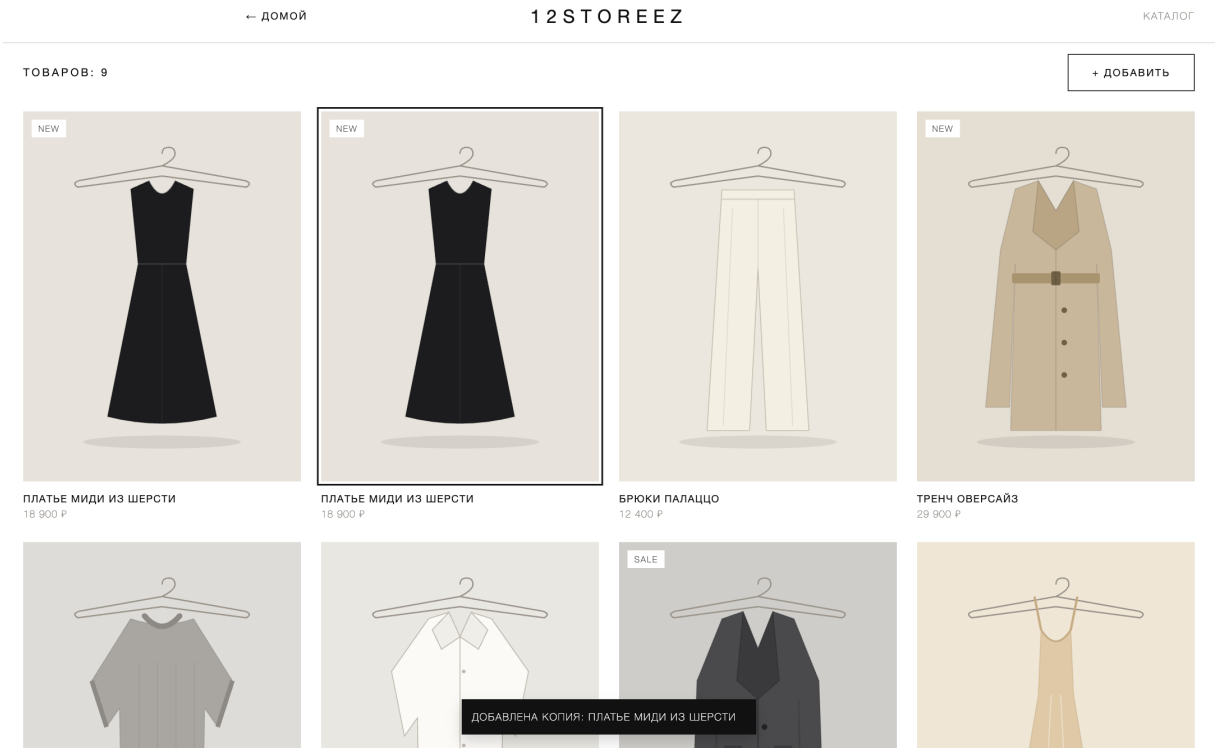


Рисунок 16 – добавление копии первой карточки

Удаление выполняется крестиком в углу карточки (появляется при наведении) либо кнопкой «Удалить» на странице товара. Обработчик вызывает `store.remove(id)`, перерисовывает карусель и сетку и показывает уведомление «Карточка удалена» (рис. 17).

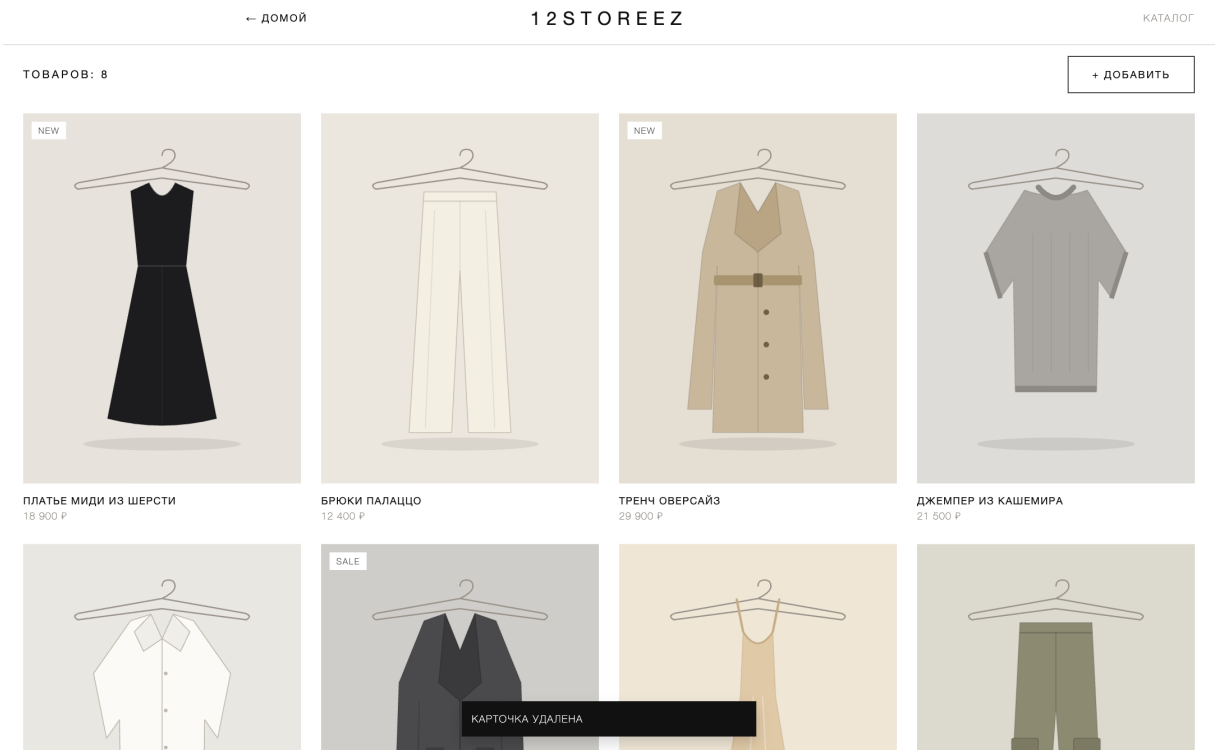


Рисунок 17 – удаление карточки

Уведомления реализованы на компоненте Bootstrap Toast (рис. 18): функция `showToast` создаёт контейнер в нижней части экрана, добавляет в него сообщение, показывает его на 2,2 секунды и удаляет элемент после скрытия.

components/toast/index.js

```
// Всплывающее уведомление (Bootstrap Toast): подтверждает добавление / удаление
export function showToast(message) {
  let box = document.getElementById('toast-box');
  if (!box) {
    document.body.insertAdjacentHTML('beforeend',
      '<div id="toast-box" class="toast-container position-fixed bottom-0 start-50 translate-middle-x p-3"></div>'
    box = document.getElementById('toast-box');
  }
  box.insertAdjacentHTML('beforeend',
    `<div class="toast s-toast" role="alert"><div class="toast-body">${message}</div></div>`);
  const el = box.lastElementChild;
  el.addEventListener('hidden.bs.toast', () => el.remove());
  new bootstrap.Toast(el, { delay: 2200 }).show();
}
```

Рисунок 18 – функция showToast на основе Bootstrap Toast

Карусель карточек

Дополнительно к заданию реализован компонент CarouselComponent (рис. 19) на основе Bootstrap Carousel. Он показывает блок «Новинки»: первые восемь товаров по четыре карточки в слайде. Метод `getHTML()` делит список на группы и формирует слайды, стрелки и индикаторы, метод `render()` вставляет карусель и вешает на карточки обработчик клика для перехода на страницу товара. Вид второго слайда показан на рисунке 20. После удаления товара карусель перерисовывается.

```
// Карусель из карточек (Bootstrap Carousel): в каждом слайде по 4 карточки
export class CarouselComponent {
  constructor(parent) { this.parent = parent; }

  getHTML(items, title = 'Новинки', perSlide = 4) {
    const groups = [];
    for (let i = 0; i < items.length; i += perSlide) groups.push(items.slice(i, i + perSlide));

    const slides = groups.map((group, i) => `
      <div class="carousel-item ${i === 0 ? 'active' : ''}">
        <div class="row g-4 px-5">
          ${group.map((p) => `
            <div class="col-6 col-lg-3">
              <div class="s-mini" data-id="${p.id}">
                <div class="photo"><img src="${p.image}" alt="${p.title}"></div>
                <div class="name">${p.title}</div>
                <div class="price">${p.price.toLocaleString('ru-RU')} ₽</div>
              </div>
            </div>`).join('')}
        </div>
      </div>`).join('');

    const dots = groups.length > 1
      ? `<div class="carousel-indicators">${groups.map((_, i) => `
          <button type="button" data-bs-target="#new-carousel" data-bs-slide-to="${i}" ${i === 0 ? 'class="active"' : ''}><
      : '';
    const arrows = groups.length > 1
      ? `<button class="carousel-control-prev" type="button" data-bs-target="#new-carousel" data-bs-slide="prev"><span clas
         <button class="carousel-control-next" type="button" data-bs-target="#new-carousel" data-bs-slide="next"><span clas
      : '';

    return `
      <section class="s-carousel">
        <p class="s-title mb-3">${title}</p>
        <div id="new-carousel" class="carousel slide">
          <div class="carousel-inner">${slides}</div>
          ${arrows}
          ${dots}
        </div>
      </section>`;
  }

  // onOpen получает id карточки; повторный вызов полностью перерисовывает карусель
  render(items, onOpen) {
    this.parent.innerHTML = items.length ? this.getHTML(items) : '';
    this.parent.querySelectorAll('.s-mini').forEach((el) =>
```

Рисунок 19 – компонент CarouselComponent



Рисунок 20 – второй слайд карусели «Новинки»

Страница «подробнее»

При нажатии на карточку выполняется переход на страницу товара (рис. 21). Идентификатор берётся из адреса `#/product/3`. Страница ProductPage (рис. 22) находит товар в модели и отображает ProductComponent: изображение, категорию, название, цену, размеры, цвет, состав и артикул, а также кнопки «Назад» (возвращает в каталог) и «Удалить». Если товар не найден, выводится сообщение «Товар не найден». Кнопка «Домой» в шапке возвращает на титульную страницу.

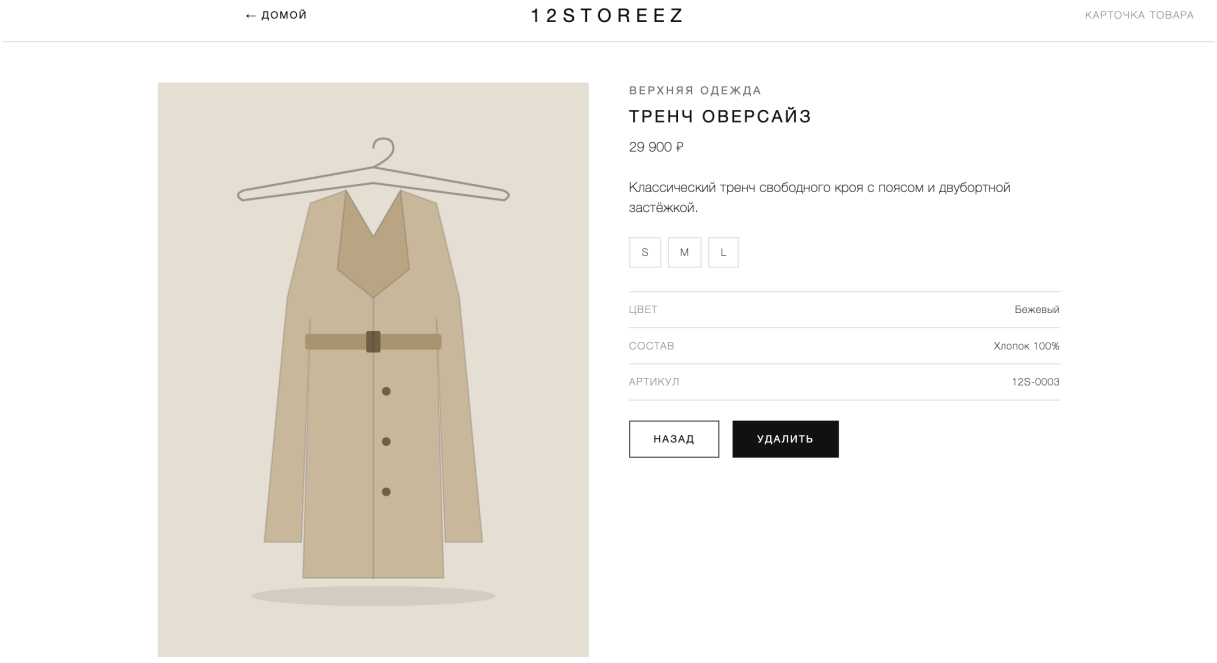
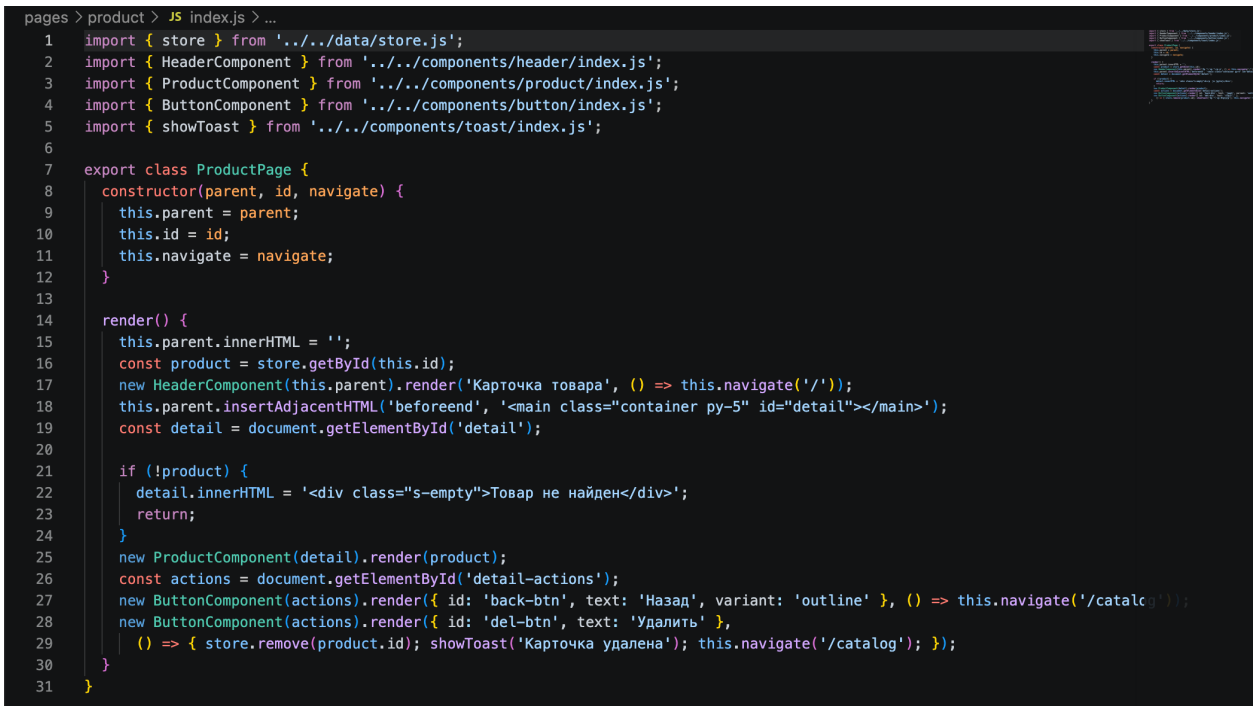


Рисунок 21 – страница «подробнее»



```
pages > product > JS index.js > ...
import { store } from '../../data/store.js';
import { HeaderComponent } from '../../components/header/index.js';
import { ProductComponent } from '../../components/product/index.js';
import { ButtonComponent } from '../../components/button/index.js';
import { showToast } from '../../components/toast/index.js';

export class ProductPage {
  constructor(parent, id, navigate) {
    this.parent = parent;
    this.id = id;
    this.navigate = navigate;
  }

  render() {
    this.parent.innerHTML = '';
    const product = store.getById(this.id);
    new HeaderComponent(this.parent).render('Карточка товара', () => this.navigate('/'));
    this.parent.insertAdjacentHTML('beforeend', '<main class="container py-5" id="detail"></main>');
    const detail = document.getElementById('detail');

    if (!product) {
      detail.innerHTML = '<div class="s-empty">Товар не найден</div>';
      return;
    }
    new ProductComponent(detail).render(product);
    const actions = document.getElementById('detail-actions');
    new ButtonComponent(actions).render({ id: 'back-btn', text: 'Назад', variant: 'outline' }, () => this.navigate('/catalog'));
    new ButtonComponent(actions).render({ id: 'del-btn', text: 'Удалить' },
      () => { store.remove(product.id); showToast('Карточка удалена'); this.navigate('/catalog'); });
  }
}
```

Рисунок 22 – страница ProductPage

Титульная страница

Страница HomePage (рис. 23) рисуется методом `render()`: в корневой элемент вставляется разметка с названием бренда, а в подготовленное место `hero-btn` ставится кнопка «Смотреть каталог», обработчик которой переходит на `#/catalog`.

pages > home > JS index.js > ...

```
import { ButtonComponent } from '../../components/button/index.js';

// Титульная страница с названием бренда; кнопка «Домой» ведёт сюда
export class HomePage {
  constructor(parent, navigate) {
    this.parent = parent;
    this.navigate = navigate;
  }

  render() {
    this.parent.innerHTML = `
      <section class="s-hero">
        <p class="s-hero-top">Женская одежда · Коллекция осень – зима</p>
        <h1 class="s-hero-logo">12Storeez</h1>
        <p class="s-hero-sub">Каталог одежды</p>
        <div id="hero-btn"></div>
        <p class="s-hero-bottom">Лабораторная работа №3</p>
      </section>`;
    new ButtonComponent(document.getElementById('hero-btn')).render(
      { id: 'to-catalog', text: 'Смотреть каталог' },
      () => this.navigate('/catalog'));
  }
}

```

Рисунок 23 – страница HomePage

Просмотр GET-запроса в Network

Для демонстрации того, что страница формируется на стороне клиента, в инструментах разработчика браузера (вкладка Network) рассмотрен GET-запрос на получение HTML-документа. На рисунке 24 показан список запросов: первым загружается index.html (тип document), затем стили, скрипты и модули – в колонке Initiator видно, какой файл запросил каждый ресурс.

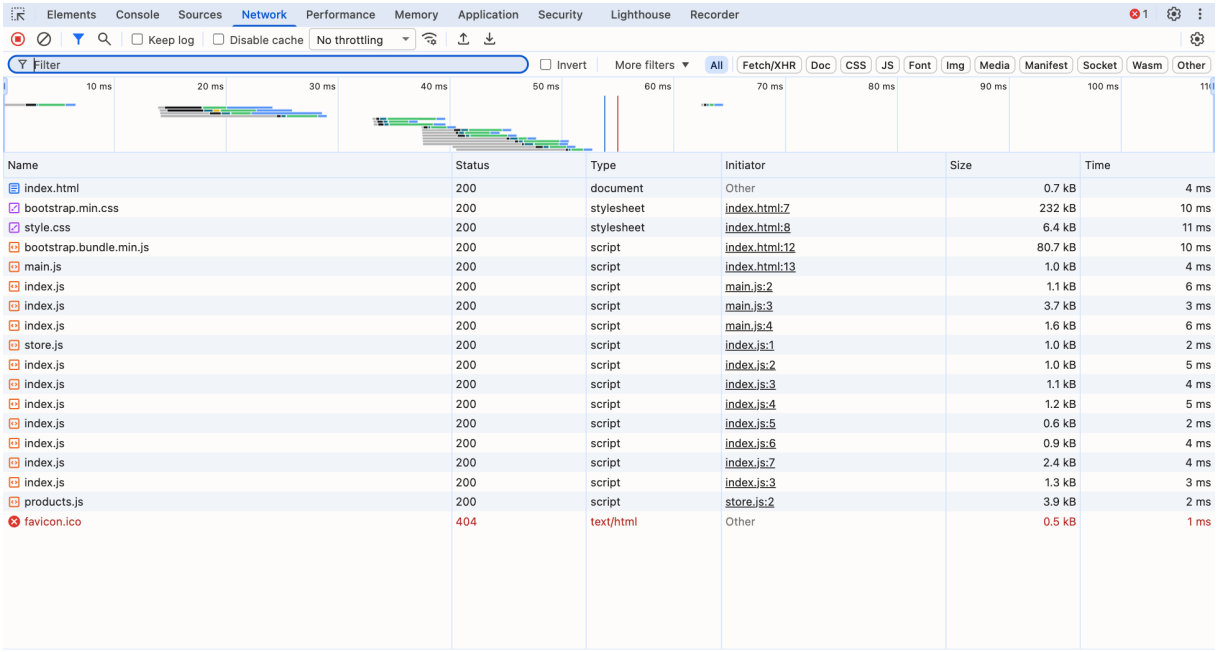


Рисунок 24 – вкладка Network: список запросов

На вкладке Headers (рис. 25) видны адрес запроса, метод GET и код ответа 200 OK.

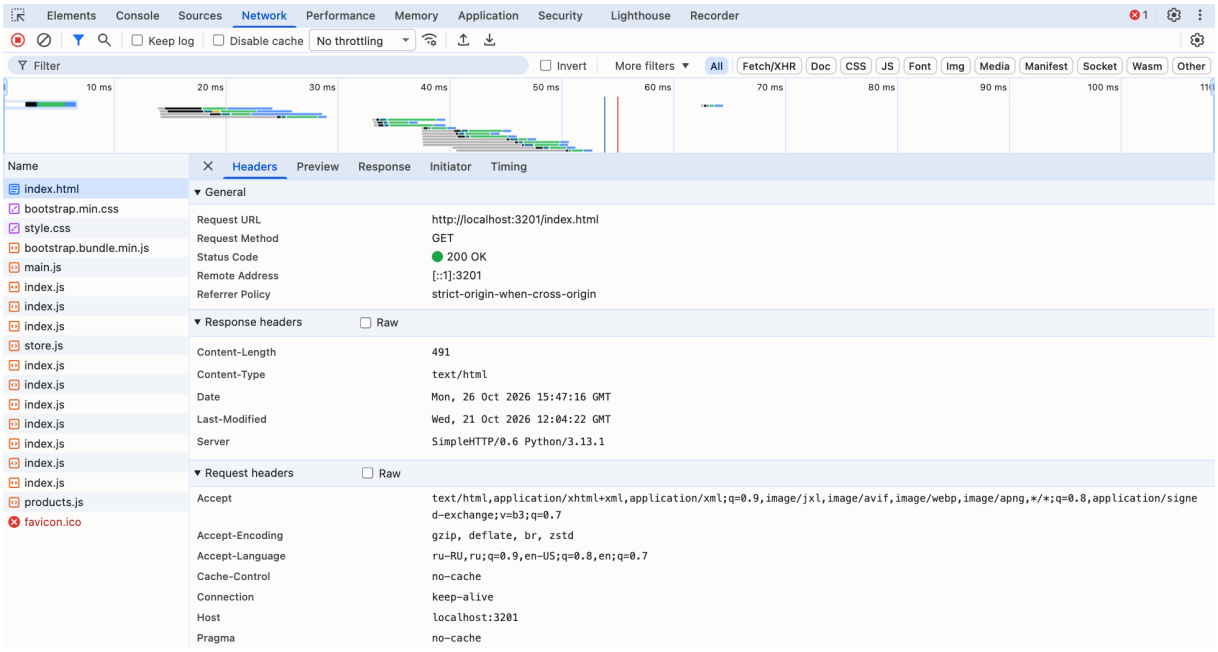


Рисунок 25 – вкладка Headers: метод GET, код 200 OK

На вкладке Response (рис. 26) показано содержимое ответа – исходный файл index.html. В нём присутствует только пустой корневой элемент `<div id="root"></div>` и подключения скриптов. Карточки, шапка, карусель и фильтры в нём отсутствуют: они создаются позже скриптом main.js после загрузки модулей.

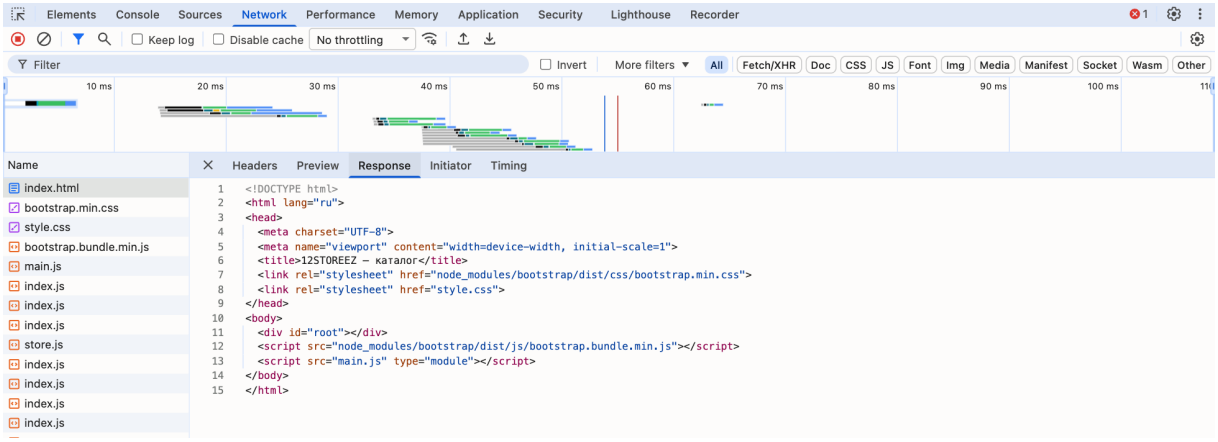


Рисунок 26 – вкладка Response: содержимое index.html

Приложение адаптировано под экран мобильного устройства (рис. 27): сетка карточек перестраивается в две колонки.

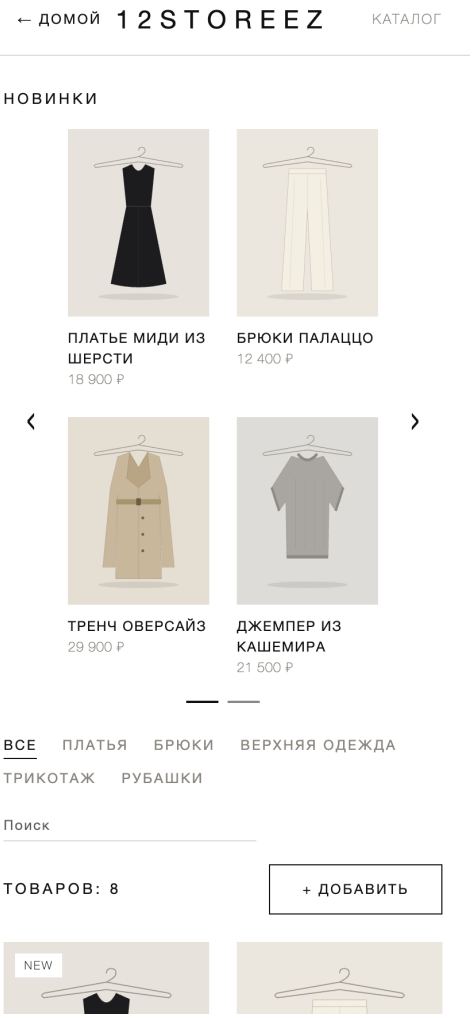


Рисунок 27 – мобильная версия

КОНТРОЛЬНЫЕ ВОПРОСЫ

Опишите структуру вашего проекта

Точка входа – index.html с корневым элементом root. Файл main.js выполняет роль роутера. В каталоге data находятся mock-коллекция products.js и хранилище store.js. В каталоге components лежат компоненты интерфейса: button, header, filters, product-card, product, carousel и toast. В каталоге pages находятся страницы home, main и product. Каталог images содержит иллюстрации товаров. Файлы package.json и package-lock.json описывают зависимости (Bootstrap), папка node_modules содержит установленные пакеты, style.css – собственные стили.

Что такое MVC

MVC (Model–View–Controller) – шаблон проектирования, разделяющий приложение на три части: Model (модель) хранит данные и логику работы с ними; View (представление) отвечает за отображение; Controller (контроллер) обрабатывает действия пользователя и обновляет модель и представление. В проекте моделью является data/store.js с коллекцией products.js, представлением – классы в каталоге components, контроллером – страницы из pages и роутер main.js.

Что такое роутинг

Роутинг – сопоставление адреса (URL) с отображаемой страницей или обработчиком. В одностраничном приложении переключение выполняется без перезагрузки: роутер отслеживает изменение адреса (в проекте – хэша в location.hash через событие hashchange) и рендерит нужную страницу.

Ключевые этапы рендера

Браузер запрашивает HTML-документ (GET), разбирает его и строит дерево DOM. Параллельно загружаются стили и скрипты; из CSS строится CSSOM. DOM и CSSOM объединяются в дерево отображения (render tree),

затем выполняется расчёт геометрии элементов (layout), отрисовка (paint) и композиция слоёв (compositing). Когда JavaScript изменяет DOM (в проекте – методом insertAdjacentHTML), часть этих этапов выполняется повторно.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы изучены основы работы с Node.js и npm, установлена и подключена библиотека Bootstrap. Разработано веб-приложение «Каталог одежды» в стиле сайта 12STOREEZ, данные для которого берутся из mock-коллекции.

Реализованы титульная страница, список карточек с фильтрацией по категориям и поиском, страница подробной информации, кнопки добавления (копия первой карточки) и удаления карточки, кнопка «Домой» в шапке обеих страниц. Дополнительно добавлены карусель «Новинки» и всплывающие уведомления на компонентах Bootstrap. Интерфейс построен из переиспользуемых компонентов-классов, а переключение страниц выполняет простой роутер на основе хэша адресной строки. Рассмотрены шаблон MVC, принципы роутинга, этапы рендера страницы и работа GET-запроса в инструментах разработчика.

ЛИТЕРАТУРА

1. Сайт 12STOREEZ [Электронный ресурс]. – URL: https://12storeez.com/ (дата обращения: 26.10.2026).
2. Лекции по курсу и методические указания по выполнению лабораторных работ [Электронный ресурс]. – URL: https://github.com/iu5git/JavaScript (дата обращения: 26.10.2026).
3. Документация Bootstrap 5.2 [Электронный ресурс]. – URL: https://bootstrap-4.ru/docs/5.2/ (дата обращения: 26.10.2026).
4. Node.js [Электронный ресурс]. – URL: https://nodejs.org (дата обращения: 26.10.2026).